

4th Place Solution - Wow! - Code Sharing

Rank	4
Team	Manuel Campos
Author	Manuel Campos
Topic ID	549603
Version	Original

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549603>
Retrieved with Kaggle CLI on 2026-07-03.

Hi everyone,

This was an amazing result for me, I'm very happy!

My 4th position solution was a "scary" combination of basically notebooks shared by other kagglers here (Kaggle Learning) well adjusted to the score offered by the Public Leaderboard.

My second submission was to follow a stacking strategy (0.420 public-0.427 private) relying on cv (5gkf 0.407221) but this time overfitting to the Public LB was the way to go, at least here.

From the results, it's possible that train and test didn't follow a similar distribution and it also seems that the 35%-65% split was a simple random sampling process

It's very late in European time so I'll be waiting, as soon as possible, to edit this thread again and express in more detail all my gratitude to the contributors of this success and share the solution code.

Thanks,

UPDATED EDIT: 2024/12/06 16:50 (CET)

Acknowledgments

I would like to thank the Host [@dennissoemers](#) and the Kaggle Staff [@ashleychow](#) [@sohier](#) [@mylesoneill](#) for organizing this competition and congratulate them for an excellent management. As on other occasions, I would like to thank Kaggle for the resources it makes available to us.

The most important thing here is to cite the kagglers and code that have constituted the basic source of my final solution. Huge thanks to you all. I include some information that may be useful to follow this thread.

(w/o priority order)

Credits	Code	Alias	Public LB	Private LB
@yekenot	MCTS: DeepTables NN -Version16 (*)	demido v	0.428	0.435
@yunsuxiaozi	MCTS Starter -Version15	yunsuxiaozi	0.427	0.437
@yunsuxiaozi	MCTS Simple Yunbase -Version6	yunbase	0.427	0.437
@hideyukizushi	MCTS - OOF Predictions as Features-HP tune[.427] -Version2	yukiZ	0.427	0.436
@verracodeguacas	Matryoshka embeddings & training -Version15	matryoshka	0.429	0.438
@ghulamhiader	UM Game playing -Version28	ghulam	0.430	0.439
@longgggl	notebook MCTS -Version25	longgggl	0.434	0.442
@xiaoleilian	sklearn pipelines feat eng + ensemble -Version2	xiaolei	0.434	0.440
@litsea		tatiana	0.434	0.442

Credits	Code	Alias	PublicLB	PrivateLB
	MCTS - Baseline + FE - LGBM - Version3			
@ambros	MCTS EDA which makes sense - Version4	ambros	0.437	0.443
@martinapreusse	MCTS - Stacked CatBoost - Version1	martpreusse	0.433	0.435
(*) @peilwang for his contribution fix DeepTables and discussion: Fix DeepTables Save & Load Error				

Solution with Code

Take a look at this link to the original notebook:

- [MCTS-4th-Place-Solution-Private-LB](#)

, I have linked a new code well referenced and with all the inputs (datasets and notebooks) shared from my private workspace. I mean to all the preparatory adaptations of the original public notebooks with a view to final inference.

[MCTS-4th-Place-Solution-Referenced-Workspace](#)

Overview

I started this competition at the beginning of October, intermittently. The first thing I did was to use [@andreasbis](#) notebook: [MCTS | OOF Predictions as Features](#). Modify some parameters of the gradient boosting models and see how it performed. Without success in making progress with it, after two weeks I did succeed in managing an ensemble with the two highest ranked public notebooks (aka "yukiZ" and "yunbase", both with 0.427 publicLB) (several weeks later this combination was temporarily shared by [@konstantinboyko](#) generating controversy among some members of the community). The result (0.423) made me continue in that direction adding 2 nnets like "demidov" to reach 0.422 (top20 then).

Do not follow as a rule to start any competition this way. The danger is that if you succeed quickly, your high LB ranking and the comfort of not thinking too much may seduce you to continue the same route. Most of the time, this is not the best path. But I continued, like this until 10 days before the Final Submission Deadline with a score of 0.415 (top2) and that was one of the submissions I selected.

The rest of the days I dedicated to setting up a solution with a stacking-strategy that would balance the high risk of the previous submission, totally overfitted to the publicLB signals.

Code	CV	PublicLB	PrivateLB	Private-Rank
Submission2-Stacking-Solution (*)	5gkf: 0.407221	0.42005	0.42711	20th
(*) Uses some weights optimized from this notebook				

I have done very little or no work here as an original contribution to the public versions. As you can imagine, there is a lot of diversity in all things between so many solutions I have used.

Cascade Merging and Manual Regression with Clip

I built the ensemble in a cascading fashion. Starting from an initial base, I added sequentially other solutions one by one, overfitting each step to the public leaderboard score. The code below is the final snapshot with scores Public:0.41521 and Private:0.42192.

Unfortunately, I cannot show you a detailed progression of the cascade scores here, because during its development I also changed the order of the steps or manipulated some of the already set steps (for example extending demidov(cat) in demidov(cat) and demidov(cat_seed)). To do this we would have to replicate each waterfall and its score (which is exceeded here). Similarly, I don't see much point in providing CV results (with the oofs) since this is not going to guide any selection.

This is the chunk:

```
merging_sub = 0.400*yukiZ + 0.400*yunsuxiaozi + 0.100*demidov_nnet +
0.100*demidov_nnet_seed
merging_sub = 1.200*merging_sub - 0.200*matryoshka_lighgbm
merging_sub = 0.700*merging_sub + 0.300*ghulam
merging_sub = 1.250*merging_sub - 0.250*xiaolei
merging_sub = 0.750*merging_sub + 0.250*longggl
merging_sub = 1.200*merging_sub - 0.200*tatiana
merging_sub = 1.100*merging_sub - 0.050*demidov_cat - 0.050*demidov_cat_seed
merging_sub = 1.050*merging_sub - 0.050*ambrosm
merging_sub = 0.850*merging_sub + 0.150*yunbase_seed
merging_sub = 1.250*merging_sub - 0.250*yunbase_seed_nontext
merging_sub = 0.900*merging_sub + 0.100*martpreusse
```

The evaluation at each step involved a manual multiplier (like b in $y=a+bX$) and an adder (like a in $y=a+bX$). "a" was not very relevant and I barely played with it. "b" had a more relevant effect although I only stressed it occasionally once I had accumulated several steps in the cascade. Finally, everything ends with a slight clip at the ends near -1 and 1.

```
merging_sub = np.clip(merging_sub*1.250, -0.98, 0.98)
merging_sub = np.where(merging_sub<0, merging_sub+0.005, merging_sub)
merging_sub = np.where(merging_sub>0, merging_sub-0.005, merging_sub)
merging_sub = np.clip(merging_sub, -0.98, 0.98)
```

My wish that...

I recognize that this is not a complex and brilliant solution but I would be satisfied if the host could extract some utility or if at least some kaggler could do so. At the very least, I suppose it can serve as a "RMSE reduction function" of the strongest notebooks shared by the community in this competition.

That's all

Regards